

Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ

по лабораторной работе №4

По дисциплине: «Аппаратное обеспечение интеллектуальных систем»

Выполнил:

Студент группы 421702 Дожди́ков А. И.

Проверил:

Захаров В. В.

Минск 2026

1 Индивидуальное задание

1.1 Вариант задания

- **Задание 1:** Синтез комбинационной схемы с несколькими выходами: одноразрядный двоичный вычитатель на 3 входа (ОДВ-3) с представлением выходных функций в СДНФ.
- **Задание 2:** Синтез преобразователя тетрад десятично-двоичного кода 8421 в код $8421 + n$ как устройства с не полностью определенными функциями. Вариант: $n = 5$.

1.2 Формулировка задачи

Разработать и проверить программу, выполняющую автоматизированный синтез логических схем. Программа должна осуществлять:

1. Анализ заданного варианта.
2. Составление таблицы истинности, описывающей поведение комбинационных схем.
3. Получение совершенных нормальных форм (СДНФ) для выходных функций.
4. Минимизацию функций с учетом избыточных наборов.
5. Вывод аналитического представления синтезированных логических уравнений для построения схем.

2 Результаты выполнения программы

2.1 Задание 1. Синтез одноразрядного двоичного вычитателя (ОДВ-3)

Согласно описанию объекта синтеза, ОДВ-3 имеет 3 входа (цифра уменьшаемого x_1 , цифра вычитаемого x_2 , заем из младшего разряда x_3) и

2 выхода (разность d_i , заем в старший разряд b_{i+1}).

На основании программной генерации получена следующая таблица истинности:

x_1	x_2	x_3	d_i	b_{i+1}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Программно сгенерированные функции в СДНФ:

$$d_i = (\overline{x_1} \cdot \overline{x_2} \cdot x_3) \vee (\overline{x_1} \cdot x_2 \cdot \overline{x_3}) \vee (x_1 \cdot \overline{x_2} \cdot \overline{x_3}) \vee (x_1 \cdot x_2 \cdot x_3) \quad (1)$$

$$b_{i+1} = (\overline{x_1} \cdot \overline{x_2} \cdot x_3) \vee (\overline{x_1} \cdot x_2 \cdot \overline{x_3}) \vee (\overline{x_1} \cdot x_2 \cdot x_3) \vee (x_1 \cdot x_2 \cdot x_3) \quad (2)$$

Результат раздельной алгоритмической минимизации (МДНФ):

$$d_i = x_1 \oplus x_2 \oplus x_3 \quad (3)$$

$$b_{i+1} = (\overline{x_1} \cdot x_2) \vee (\overline{x_1} \cdot x_3) \vee (x_2 \cdot x_3) \quad (4)$$

Анализ показывает, что реализация ОДВ-3 с использованием раздельных функций потребует ориентировочно 28 транзисторов.

Для дополнительной аппаратной оптимизации был применен метод совместного синтеза устройства с несколькими выходами. Выразим функцию d_i через функцию заема b_{i+1} , используя сложение функции b_{i+1} с логическим произведением $(x_1 \cdot \overline{x_2} \cdot \overline{x_3})$ для нейтрализации ненужной конъюнкции. В результате получаем оптимизированные уравнения:

$$b_{i+1}^* = b_{i+1} \vee (x_1 \cdot \overline{x_2} \cdot \overline{x_3}) \quad (5)$$

$$d_i = [b_{i+1} \vee (x_1 \cdot \overline{x_2} \cdot \overline{x_3})] \cdot (\overline{x_1} \vee x_2 \vee x_3) \quad (6)$$

Данные математические преобразования позволяют использовать общие логические элементы для двух выходов, что сокращает аппаратные затраты до 22 транзисторов. Принципиальная схема ОДВ-3, построенная по минимизированным формулам (5) и (6), представлена на Рисунке 1.

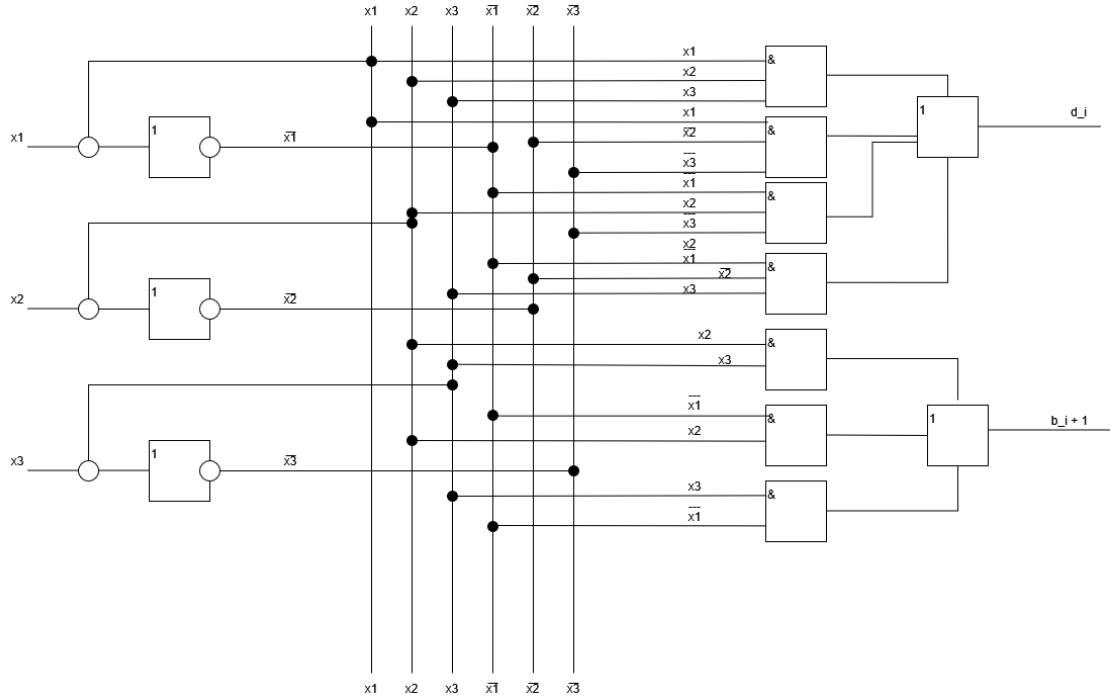


Рис. 1: Логическая схема синтезированного ОДВ-3

2.2 Задание 2. Синтез преобразователя кода 8421 в $8421 + 5$

Устройство имеет 4 входа (x_4, x_3, x_2, x_1) и 4 выхода (y_4, y_3, y_2, y_1). Так как возможных наборов 16, а десятичных цифр 10, наборы со значениями от 10 до 15 ($1010_2 \dots 1111_2$) являются избыточными. Они используются алгоритмом для доопределения функций.

Таблица истинности для преобразователя $8421 \rightarrow 8421 + 5$:

Результаты логической минимизации с учетом массива избыточных наборов:

$$y_4 = x_4 \vee (x_3 \cdot x_2) \vee (x_3 \cdot x_1) \quad (7)$$

$$y_3 = (\overline{x_3} \cdot \overline{x_2} \cdot \overline{x_1}) \vee (\overline{x_4} \cdot \overline{x_3} \cdot x_2) \vee (x_4 \cdot \overline{x_1}) \quad (8)$$

$$y_2 = (\overline{x_2} \cdot x_1) \vee (x_2 \cdot \overline{x_1}) \quad (9)$$

№	x_4	x_3	x_2	x_1	y_4	y_3	y_2	y_1
0	0	0	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0
2	0	0	1	0	0	1	1	1
3	0	0	1	1	1	0	0	0
4	0	1	0	0	1	0	0	1
5	0	1	0	1	1	0	1	0
6	0	1	1	0	1	0	1	1
7	0	1	1	1	1	1	0	0
8	1	0	0	0	1	1	0	1
9	1	0	0	1	1	1	1	0
10-15	-	-	-	-	-	-	-	-

$$y_1 = \overline{x_1} \quad (10)$$

Принципиальная схема преобразователя кода, синтезированная на основании полученных минимизированных тупиковых форм, представлена на Рисунке 2.

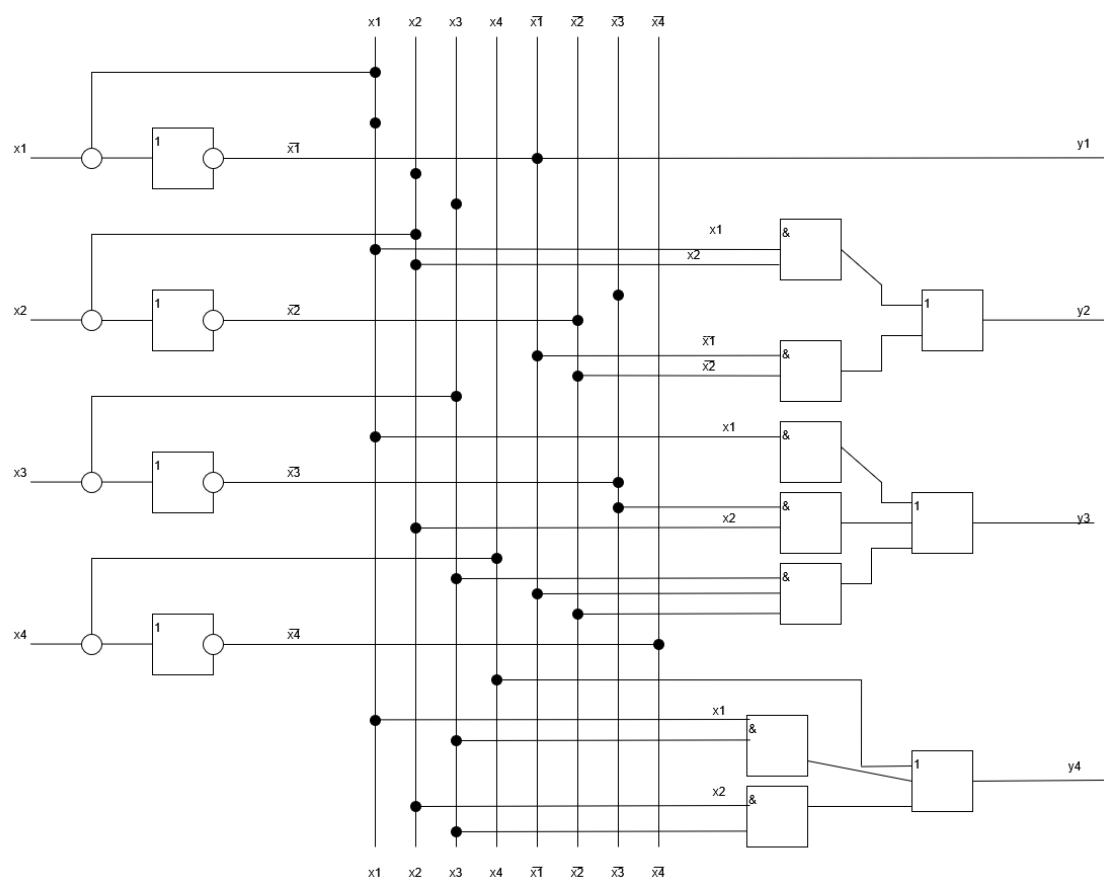


Рис. 2: Логическая схема преобразователя кода $8421 \rightarrow 8421 + 5$

3 Вывод

Программная реализация синтеза комбинационных схем (на примере ОДВ-3 и преобразователя кода) позволила на практике изучить процесс аппаратной оптимизации логических функций. Практически доказано, что использование избыточных наборов и совместная минимизация функций с несколькими выходами позволяет существенно сократить объем необходимого оборудования.

Теоретической базой для преобразований послужили следующие понятия:

- **Синтез комбинационных схем** — процесс построения логической схемы, не содержащей элементов памяти, на основе математических выражений (СДНФ или СКНФ), полученных из таблицы истинности устройства.
- **Синтез устройств с несколькими выходами** — алгоритм аппаратной оптимизации, основанный на возможности выражать одни выходные логические функции через другие за счет выявления общих конституэнт.
- **Не полностью определенные функции** — логические функции, имеющие избыточные комбинации аргументов, которые физически не могут появиться на входе устройства. Целенаправленное доопределение таких наборов нулями или единицами позволяет добиться максимального сокращения итоговой формулы.
- **Правило склеивания** — базовый алгебраический инструмент минимизации, позволяющий сокращать переменные: $(A \cdot x) \vee (A \cdot \bar{x}) = A$.

Сравнительный анализ изученных методов минимизации логических функций:

1. **Расчетный (алгебраический) метод:** Выполняется алгебраически через применение законов логики. Не требует графических построений, но при большом количестве переменных метод становится трудоемким.

2. **Расчетно-табличный метод (Квайна-Мак-Класки):** Состоит из нахождения простых импликант и построения таблицы покрытий. Метод полностью алгоритмизирован и идеально подходит для программной реализации, что и было успешно применено в написанном коде на Python.
3. **Табличный метод (карты Вейча-Карно):** Самый быстрый и интуитивно понятный визуальный метод. Позволяет мгновенно записать тупиковую форму. Однако метод теряет наглядность при $n > 4$ и сложен для алгоритмизации.

Приложение А. Исходный код программы

```
1 import itertools
2 from sympy.logic import SOPform
3 from sympy import symbols
4
5 x1, x2, x3, x4 = symbols('x1 x2 x3 x4')
6
7 def generate_perfect_forms(vars_sym, minterms, maxterms):
8     sdnf_terms = []
9     for term in minterms:
10         vars_str = [str(vars_sym[i]) if val == 1 else f"~{
11             vars_sym[i]}" for i, val in enumerate(term)]
12         sdnf_terms.append("(" + " * ".join(vars_str) + ")")
13     sdnf_str = " + ".join(sdnf_terms) if sdnf_terms else "0"
14     return sdnf_str
15
16 def task1_3input_device(device_type="          -3"):
17     print(f"\n{'='*50}\n          1.
18         : {device_type}\n{'='*50}")
19
20     inputs = list(itertools.product([0, 1], repeat=3))
21     minterms_out1, minterms_out2 = [], []
22
23     for x1_val, x2_val, x3_val in inputs:
24         out1 = x1_val ^ x2_val ^ x3_val
25         out2 = int((not x1_val and x2_val) or (not x1_val and
26             x3_val) or (x2_val and x3_val))
27
28         term = [x1_val, x2_val, x3_val]
29         if out1 == 1: minterms_out1.append(term)
30         if out2 == 1: minterms_out2.append(term)
31
32     vars_sym = [x1, x2, x3]
33     print(f"OUT1          : {generate_perfect_forms(vars_sym,
34         minterms_out1, [])}")
35     print(f"OUT1          (          ): {
36         SOPform(vars_sym, minterms_out1)}\n")
37     print(f"OUT2          : {generate_perfect_forms(vars_sym,
38         minterms_out2, [])}")
```

```

32     print(f"OUT2                                     (          ): {
          SOPform(vars_sym, minterms_out2)}\n")
33
34 def task2_code_converter(n_variant=5):
35     print(f"\n{'='*50}\n                                2.
                                                : 8421 -> 8421 + {
          n_variant}\n{'='*50}")
36     inputs = list(itertools.product([0, 1], repeat=4))
37     minterms = {i: [] for i in range(4)}
38     dontcares = []
39
40     for idx, (x4_val, x3_val, x2_val, x1_val) in enumerate(
        inputs):
41         if idx > 9:
42             dontcares.append([x4_val, x3_val, x2_val, x1_val])
43             continue
44
45         val = (idx + n_variant) % 16
46         y_bits = [int(b) for b in format(val, '04b')]
47
48         for i in range(4):
49             if y_bits[i] == 1:
50                 minterms[i].append([x4_val, x3_val, x2_val,
                    x1_val])
51
52     vars_sym = [x4, x3, x2, x1]
53     for i in range(4):
54         minimized = SOPform(vars_sym, minterms[i], dontcares)
55         print(f"y{4-i}
                                                : {minimized}")
56
57 if __name__ == "__main__":
58     task1_3input_device("          -3")
59     task2_code_converter(n_variant=5)

```